

B³ — Bug Bounty Broker

Claude Code Project Context & Architecture

1. WHAT THIS IS

B³ is a sovereign agent running on EigenCloud that acts as a **neutral, TEE-sealed intermediary** for smart contract bug bounties. It lets security researchers **prove a vulnerability exists and get paid without revealing it first**, eliminating the trust asymmetry that plagues every existing bounty platform.

The agent will be deployed as an EigenCompute app, presented at Eigen Labs' Private Preview Demo Day on **May 12, 2026**.

2. WHY THIS MATTERS — THE PROBLEM

The trust asymmetry in bug bounties

Today's flow: researcher reveals full vulnerability → company decides if it's real → company decides severity → company decides payment. The company has full information advantage at every step.

Documented failures (use these stats in writeups/demos)

- **Injective (March 2026)**: Researcher found \$500M vulnerability, disclosed via Immunefi. After 3 months silence, offered \$50K (10% of advertised max \$500K). Researcher went public.
- **HackerOne ghosting (Jan 2026)**: Veteran researcher (20+ disclosures) submitted reports Oct 30, 2025. Messaged Nov 14, Nov 19, Dec 15, emailed Dec 15, DM'd employee Dec 22. Zero response until a journalist asked.
- **Silent patching**: Researchers report bugs dismissed as "informative" on HackerOne, only to find the company silently patches the vulnerability without paying.
- **Slack**: Paid \$1,750 for a vulnerability a researcher spent significant effort finding.
- **HackerOne IBB paused (April 2026)**: Entire Internet Bug Bounty program suspended — AI finds bugs faster than maintainers fix them; Node.js bounty program killed as collateral.

Market size

- Bug bounty platforms market: **\$1.5B in 2024**, projected \$5.7B by 2033 (16% CAGR).

- ImmuneFi (Web3 leader): \$115M+ total payouts, 400+ protocols.
- Google paid \$11.8M in 2024. Meta paid \$2.3M.
- Web3 lost **\$3.1B in H1 2025** from exploits. Bounty programs claim preventing \$25B+ in potential losses.

What exists today and why it's insufficient

Platform	What it does	What's still broken
HackerOne/Bugcrowd	Human-mediated triage + payout	Company sees full report before deciding to pay. Can stall, downgrade, silently patch.
ImmuneFi	Web3 bounties, mediates disputes	Project team still decides severity and payout. 3-month delays documented.
Remedy (r.xyz)	ZK proof-of-submission timestamps	Only proves <i>when</i> you submitted, not <i>what</i> or <i>whether it's valid</i> . Company still sees report first.
Hats Finance	On-chain escrow for bounties	Verification still done by project team (incentive to downgrade).

The gap: No system verifies a vulnerability exists without revealing it to the paying party, holds funds in neutral escrow, and produces a deterministic severity assessment. B³ fills this.

3. EIGENCLOUD PLATFORM — WHAT WE HAVE

EigenCompute (our primary tool)

- Docker containers running in **Intel TDX** TEEs on Google Cloud Confidential Space.
- Encrypted memory (AES-XTS-128), hardware attestation via TDX Quote.
- Each app gets a **deterministic Ethereum wallet** via `process.env.MNEMONIC` (BIP-39, derived from app ID, stable across restarts if image digest unchanged).
- Secrets in `.env` encrypted client-side by KMS, decrypted only inside attested enclave. Variables with `_PUBLIC` suffix are visible externally.
- On-chain app registry: image digest → Ethereum address mapping, publicly auditable.

Deployment flow (ecloud CLI)

```
bash
```

```
npm install -g @layr-labs/eccloud-cli
eccloud auth generate --store
eccloud billing subscribe
eccloud compute app create --name b3-agent --language typescript
# Edit code, create Dockerfile (linux/amd64, run as root)
eccloud compute app deploy # builds, encrypts, deploys
eccloud compute app configure tls # HTTPS via Caddy inside TEE
eccloud compute app info # get public IP
eccloud compute app logs # monitor
eccloud compute app upgrade # redeploy changes
```

Dockerfile requirements

- Must be `linux/amd64`
- Must run as root
- Use EXPOSE for ports
- Bind to 0.0.0.0

Environment inside TEE

- `process.env.MNEMONIC` — BIP-39 mnemonic, only decryptable inside the TEE
- All `.env` vars available
- `_PUBLIC` suffix vars visible to external users
- TLS termination happens inside the enclave (Caddy + Let's Encrypt)

Trust model (be honest about this)

- Mainnet Alpha: developer is still trusted. Full trustless execution is roadmap.
- You transitively trust: Intel TDX silicon, Google Cloud host, Eigen Labs KMS, developer key.
- No slashing path yet if TDX is compromised. Restaking-backed security is roadmap.
- **Pitch it as "minimized, verifiable, on-chain-anchored trust" — not "trustless".**

EigenAI (deterministic LLM inference)

- Endpoint: `https://eigenai.eigencloud.xyz/v1` (OpenAI-compatible)
- Auth: `x-api-key` header
- Models: `gpt-oss-120b-f16`, `qwen3-32b-128k-bf16` (open-weight only)

- **Bit-exact deterministic:** same request → same output, verified across 10K runs, ~1.8% overhead.
- Every response includes a `signature` field (65-byte secp256k1 over request+response+model+chainID).
- Verify with `ethers.verifyMessage(message, '0x' + signature)` → recover signer → look up in KeyRegistrar contract.
- **Use for:** severity assessment, CVSS scoring, exploit analysis. Determinism means anyone can replay the same assessment.

```
python

from openai import OpenAI
client = OpenAI(
    base_url="https://eigenai.eigencloud.xyz/v1",
    default_headers={"x-api-key": API_KEY},
)
response = client.chat.completions.create(
    model="qwen3-32b-128k-bf16",
    seed=42,
    messages=[{"role": "user", "content": "..."}]
)
# response includes .signature field
```

EigenDA (data availability)

- Tamper-resistant storage with KZG commitments.
- Use for: storing signed attestations, evidence packages, audit trails.
- Not a database — use for append-only evidence records.

AgentKit (March 2026)

- Wraps EigenCompute + EigenAI + wallet + persistent state for agent devs.
- Adds inference gateway routing to frontier models (Claude, GPT) for non-deterministic tasks.
- Persistent encrypted state across TEE restarts.

x402 Payment Protocol

- HTTP 402-based payment. Client GETs resource → server returns 402 with payment requirements → client signs EIP-3009 USDC transfer → retries with `Authorization` header

→ server verifies via Facilitator.

- Works on Base, Ethereum, Polygon, Arbitrum, Solana.
- Reference implementation: `Layr-Labs/compute-escrow-privy` repo (x402 + escrow + Privy auth).
- **Use for:** companies paying per-bounty to get severity attestations and unlock reports.

Credits available

- \$500 initial, up to \$1500 total for compute + inference.
- Can expense up to \$200 for AI dev tools (Claude Code, etc).

4. B³ ARCHITECTURE

Core concept

Separate vulnerability verification from vulnerability disclosure.

Researcher	B ³ Agent (TEE)	Company
-- submit encrypted PoC ----->		
	-- fork chain, replay exploit -->	
	-- assess CVSS via EigenAI ----->	
	-- sign attestation ----->	
	-- publish: "Critical vuln,	
	CVSS 9.1, \$X at risk,	
	verified by app 0xABC" ----->	
	<-- company deposits bounty ----	
	(USDC to agent wallet)	
<-- agent releases report ----		
<-- agent releases payment ---		

System components

1. API Server (FastAPI, Python)

Primary interface. Runs inside EigenCompute TEE.

Endpoints:

- `POST /bounties` — Company registers a bounty program (target contract, chain, severity tiers, max payout per tier)
- `POST /submit` — Researcher submits encrypted PoC (encrypted to agent's public key)
- `GET /attestations/{id}` — Public attestation with signed severity, no exploit details
- `POST /deposit/{attestation_id}` — Company deposits bounty (x402 or direct USDC transfer to agent wallet)
- `GET /report/{attestation_id}` — Returns full report ONLY after deposit confirmed (gated)
- `GET /verify/{attestation_id}` — Public verification endpoint: returns signed data, image digest, instructions to verify on-chain
- `GET /programs` — List active bounty programs
- `GET /health` — Health check

2. Exploit Verification Engine

The core differentiator. Runs inside TEE.

For **smart contract bounties** (v1 focus):

1. Receive encrypted PoC (Solidity test, Foundry script, or transaction sequence)
2. Fork target chain at latest block using local Anvil/Hardhat fork
3. Execute the PoC against the fork
4. Measure: did it drain funds? How much? What access did it gain?
5. Compute funds-at-risk amount
6. All execution happens inside TEE — PoC never leaves enclave until payment

For **v1 simplification**: focus exclusively on Ethereum/EVM smart contract vulnerabilities. The PoC format is a Foundry test file that the agent runs via `forge test --fork-url`.

3. Severity Assessment (EigenAI)

After exploit verification:

1. Feed exploit results + contract code + impact analysis to EigenAI (`qwen3-32b-128k-bf16`)
2. Prompt includes CVSS 3.1 scoring rubric
3. EigenAI returns deterministic CVSS score + severity classification + reasoning
4. Response includes signature — anyone can replay this exact assessment
5. Store: `{prompt_hash, response_hash, eigenai_signature, cvss_score, severity}`

The agent produces a signed attestation:

```
json
{
  "attestation_id": "uuid",
  "target_contract": "0x...",
  "chain_id": 1,
  "severity": "CRITICAL",
  "cvss_score": 9.1,
  "funds_at_risk_usd": 500000,
  "exploit_verified": true,
  "verification_block": 19283847,
  "eigenai_signature": "0x...",
  "eigenai_model": "qwen3-32b-128k-bf16",
  "agent_signature": "0x...", // signed by TEE wallet
  "app_digest": "sha256:abc...", // Docker image digest
  "timestamp": "2026-05-05T12:00:00Z"
}
```

- `agent_signature` = `sign(keccak256(attestation_data), MNEMONIC-derived key)`
- Anyone can verify: `ethers.verifyMessage(data, agent_signature)` → should match on-chain registered app address

5. Escrow & Payment

- Company deposits USDC to the agent's deterministic wallet address
- Agent monitors for deposit matching the required bounty amount for the severity tier
- On confirmed deposit: decrypt and release full PoC report to company
- Transfer bounty minus fee (5-10%) to researcher's specified address
- Fee stays in agent wallet to pay for compute costs (self-sustainability)

6. Frontend Dashboard (React + TypeScript)

- Landing page explaining B³
- Browse active bounty programs
- Researcher submission form (upload PoC, specify target)
- Attestation viewer with live verification button
- Company dashboard: register program, view attestations, deposit

- Verification ceremony page: click to verify → recovers signer → checks on-chain → green checkmark

Tech stack

EigenCompute TEE Container:

- └─ Python 3.11+ (FastAPI)
 - └─ uvicorn (ASGI server)
 - └─ web3.py (Ethereum interaction)
 - └─ eth-account (wallet/signing)
 - └─ openai (EigenAI client)
 - └─ pydantic (data validation)
 - └─ httpx (HTTP client)
- └─ Foundry (forge) – installed in container for exploit replay
- └─ SQLite – local encrypted DB for attestation records
- └─ Caddy – TLS termination inside TEE

Frontend (separate static build, served by FastAPI or separate):

- └─ React + TypeScript + Vite
- └─ Tailwind CSS
- └─ ethers.js (signature verification)
- └─ wagmi/viem (wallet connection for company deposits)
- └─ shadcn/ui components

Dockerfile skeleton

dockerfile


```

FROM --platform=linux/amd64 python:3.11-slim
USER root
WORKDIR /app

# Install Foundry
RUN curl -L https://foundry.paradigm.xyz | bash && \
    /root/.foundry/bin/foundryup

ENV PATH="/root/.foundry/bin:${PATH}"

# Install Python deps
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy app
COPY . .

# Build frontend (if bundled)
# RUN cd frontend && npm install && npm run build

EXPOSE 3000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "3000"]

```

.env structure

```

bash

# Private (encrypted by KMS, only in TEE)
EIGENAI_API_KEY=your_eigenai_key
MNEMONIC= # auto-provided by EigenCompute

# Public (visible to users via _PUBLIC suffix)
AGENT_ADDRESS_PUBLIC=0x...
APP_DIGEST_PUBLIC=sha256:...
SUPPORTED_CHAINS_PUBLIC=1,8453,137
FEE_PERCENTAGE_PUBLIC=5
DOMAIN=b3.yourdomain.com
APP_PORT=3000

```

5. DEMO DAY STRATEGY

The 60-second verification ceremony

1. Show a deliberately vulnerable smart contract (pre-deployed to Base Sepolia)
2. Submit a PoC exploit to B³ live
3. B³ verifies the exploit on a forked chain inside TEE
4. B³ produces signed attestation with CVSS score
5. Judge clicks "Verify" on the dashboard → ethers.verifyMessage recovers address → matches on-chain app registry → green checkmark
6. Deposit test USDC → report unlocks → researcher gets paid
7. Show the contrast: "On HackerOne, this takes weeks and the company can just say no."

What to have pre-built for demo

- 2-3 intentionally vulnerable contracts (reentrancy, access control, integer overflow)
 - Pre-loaded PoC exploits ready to submit
 - A polished verification page that makes the proof ceremony visual and immediate
 - The entire flow working end-to-end on Base Sepolia testnet
-

6. PROJECT STRUCTURE

```
b3/
├── CLAUDE.md           # This file (project context)
├── agent/              # Backend (runs in EigenCompute)
│   ├── app/
│   │   ├── main.py     # FastAPI app, routes
│   │   ├── config.py   # Environment, constants
│   │   ├── models.py   # Pydantic models
│   │   ├── routes/
│   │   │   ├── bounties.py # Bounty program CRUD
│   │   │   ├── submissions.py # PoC submission + verification
│   │   │   ├── attestations.py # Attestation queries + verification
│   │   │   └── payments.py  # Deposit monitoring + payout
│   │   └── services/
│   │       ├── exploit_verifier.py # Fork chain, run PoC via Foundry
│   │       ├── severity_assessor.py # EigenAI CVSS scoring
│   │       ├── signer.py      # TEE wallet signing
│   │       ├── escrow.py      # USDC monitoring + transfers
│   │       └── crypto.py      # Encryption/decryption utils
```

```

|   |   └─ db.py                # SQLite storage
|   └─ contracts/                # Vulnerable test contracts for demo
|       └─ VulnerableVault.sol
|       └─ ReentrancyTarget.sol
|       └─ AccessControlBug.sol
|   └─ exploits/                # Demo PoC scripts (Foundry tests)
|       └─ ExploitVault.t.sol
|       └─ ExploitReentrant.t.sol
|   └─ Dockerfile
|   └─ requirements.txt
|   └─ .env.example
|   └─ foundry.toml
└─ frontend/                    # React dashboard
    └─ src/
        └─ App.tsx
        └─ pages/
            └─ Landing.tsx
            └─ Programs.tsx
            └─ Submit.tsx
            └─ Attestation.tsx # View + verify attestation
            └─ Verify.tsx      # Verification ceremony
        └─ components/
            └─ VerifyButton.tsx # The "proof button"
            └─ ...
        └─ lib/
            └─ verify.ts        # ethers.verifyMessage logic
    └─ package.json
    └─ vite.config.ts
    └─ tailwind.config.ts
└─ docs/
    └─ architecture.md          # Architecture diagram (deliverable)
    └─ product-feedback.md      # Eigen Labs feedback doc (deliverable)
└─ README.md                    # Project overview (deliverable)

```

7. BUILD ORDER (priority sequence)

Phase 1: Core verification loop (days 1-4)

1. FastAPI scaffold with health endpoint
2. TEE wallet integration (`process.env.MNEMONIC` → eth-account)

3. Exploit verification engine: accept a Foundry test file, run `forge test --fork-url` inside container, parse results
4. Signing service: sign attestation data with TEE wallet
5. SQLite storage for attestations
6. Basic `/submit` and `/attestations/{id}` endpoints

Phase 2: Severity + EigenAI (days 5-7)

1. EigenAI integration for CVSS scoring
2. Prompt engineering for deterministic severity assessment
3. Store EigenAI signature alongside agent signature
4. `/verify` endpoint returning all proof data

Phase 3: Payment flow (days 8-10)

1. USDC deposit monitoring (poll agent wallet for incoming transfers)
2. Conditional report release on confirmed deposit
3. Researcher payout (USDC transfer minus fee)
4. x402 integration if time permits (otherwise direct transfer is fine for demo)

Phase 4: Frontend + polish (days 11-14)

1. React dashboard: landing, programs, submit, attestation viewer
2. **Verification ceremony page** (highest priority UI element)
3. Demo vulnerable contracts deployed to Base Sepolia
4. Pre-built PoC exploits for live demo
5. End-to-end test of full flow

Phase 5: Deliverables (days 15-17)

1. Architecture diagram
 2. README with setup instructions
 3. Video demo recording
 4. Product feedback doc for Eigen Labs
 5. Deploy to custom domain with TLS
-

8. KEY IMPLEMENTATION NOTES

Exploit verification (the hard part)

```
python

import subprocess

async def verify_exploit(poc_path: str, fork_url: str, target_block: int) -> dict:
    """Run a Foundry test against a forked chain inside the TEE."""
    result = subprocess.run(
        [
            "forge", "test",
            "--fork-url", fork_url,
            "--fork-block-number", str(target_block),
            "--match-path", poc_path,
            "-vvvv", # verbose output for trace analysis
            "--json"
        ],
        capture_output=True, text=True, timeout=120
    )
    # Parse JSON output for pass/fail, gas used, traces
    # Extract balance changes to compute funds-at-risk
    return parse_forge_output(result.stdout, result.stderr)
```

Signing attestations

```
python

from eth_account import Account
from eth_account.messages import encode_defunct
import json, os

def sign_attestation(attestation_data: dict) -> str:
    """Sign attestation with TEE-sealed wallet."""
    mnemonic = os.environ["MNEMONIC"]
    account = Account.from_mnemonic(mnemonic)
    canonical = json.dumps(attestation_data, sort_keys=True, separators=(',', ':'))
    message = encode_defunct(text=canonical)
    signed = account.sign_message(message)
    return signed.signature.hex()
```

EigenAI severity assessment

python

```
from openai import OpenAI
```

```
SEVERITY_PROMPT = """You are a smart contract security auditor. Given the following
```

```
Respond ONLY with valid JSON:
```

```
{  
    "cvss_score": <float>,  
    "severity": "<CRITICAL|HIGH|MEDIUM|LOW|INFO>",  
    "attack_vector": "<NETWORK|ADJACENT|LOCAL|PHYSICAL>",  
    "funds_at_risk_wei": "<amount>",  
    "reasoning": "<brief explanation>"  
}
```

```
Exploit results:
```

```
{exploit_output}
```

```
Target contract:
```

```
{contract_source}
```

```
"""
```

```
async def assess_severity(exploit_output: str, contract_source: str) -> dict:
```

```
    client = OpenAI(  
        base_url="https://eigenai.eigencloud.xyz/v1",  
        default_headers={"x-api-key": os.environ["EIGENAI_API_KEY"]},  
    )
```

```
    response = client.chat.completions.create(  
        model="qwen3-32b-128k-bf16",  
        seed=42,  
        max_tokens=500,  
        messages=[  
            {  
                "role": "user",  
                "content": SEVERITY_PROMPT.format(  
                    exploit_output=exploit_output,  
                    contract_source=contract_source  
                )  
            }  
        ]  
    )
```

```
    # response has .signature for verification  
    return {
```

```
        "assessment": json.loads(response.choices[0].message.content),  
        "eigenai_signature": response.signature,
```

```
    "model": response.model,  
  }  
}
```

Verification ceremony (frontend)

typescript

```
import { verifyMessage } from 'ethers';  
  
async function verifyAttestation(attestation: Attestation): Promise<VerificationResult> {  
  // 1. Reconstruct canonical message  
  const canonical = JSON.stringify(attestation.data, Object.keys(attestation.data).sort());  
  
  // 2. Recover signer from agent signature  
  const recoveredAddress = verifyMessage(canonical, attestation.agent_signature);  
  
  // 3. Check against known app address (from on-chain registry or /health endpoint)  
  const expectedAddress = await fetch('/api/health').then(r => r.json()).then(d => d.address);  
  
  // 4. Verify EigenAI signature (replay the prompt to confirm determinism)  
  // This proves the severity assessment was produced by EigenAI, not fabricated  
  
  return {  
    valid: recoveredAddress.toLowerCase() === expectedAddress.toLowerCase(),  
    signer: recoveredAddress,  
    appDigest: attestation.app_digest,  
  };  
}
```

9. WHAT NOT TO BUILD

- Don't build a full prediction market or oracle. This is a bug bounty broker.
- Don't build a generic "personal agent." Eigen Labs explicitly said these are not useful.
- Don't overcomplicate the escrow — direct USDC transfers to/from the agent wallet are sufficient for demo. Full smart contract escrow is a v2 feature.
- Don't try to support non-EVM chains in v1. Stick to Ethereum/Base/Polygon forks.
- Don't claim "trustless" — the TEE trust model is Intel+Google+Eigen. Say "minimized trust" or "verifiable."

- Don't spend time on user auth — for demo, the agent processes submissions permissionlessly.
-

10. SUCCESS CRITERIA

The demo wins if:

1. A real exploit runs inside the TEE and produces a verifiable attestation
2. The severity assessment is deterministic (replay the same EigenAI call → same result)
3. Payment triggers report release automatically
4. A judge can click "Verify" and see cryptographic proof on-screen
5. The whole flow completes in under 60 seconds during the pitch